

```
1 // Mc32gestSpiDac.C
2 // Gestion master SPI avec slave DAC
3
4
5 #include "Mc32Delays.h"
6 #include "Mc32gestSpiFT800.h"
7 #include "Mc32SpiUtil.h"
8 #include <stdint.h>
9 #include <stdbool.h>
10 #include <stddef.h>
11 #include <stdlib.h>
12 #include "system_config.h"
13 #include "system_definitions.h"
14
15 // Initialisation de la communication SPI et du FT800
16 // -----
17 unsigned short cmd_offset = 0;
18 unsigned short dl_offset = 0;
19
20 void InitFT800(void) {
21     int i = 0;
22     // Pulse pour le démarrage
23     delay_msCt(NbMs: 20);
24     PD_TFTOff();
25     delay_msCt(NbMs: 20);
26     PD_TFTOn();
27     delay_msCt(NbMs: 20);
28
29     // Première commandes nécessaires à l'initialisation
30     // HostCommand(CLKEXT); pour EVE1
31     // HostCommand(CLK48M); pour EVE1
32
33     HostCommand(command: 0x68);
34     HostCommand(command: 0x00);
35     delay_msCt(NbMs: 300);
36
37     while (rd8(addr: REG_ID) != 0x7C);
38     while (rd16(addr: REG_CPURESET) != 0x00);
39
40     // Configure display registers - demonstration for WQVGA resolution
41     wr16(addr: REG_HCYCLE, value: 928); // number of horizontal cycles for display ↗
42     wr16(addr: REG_HOFFSET, value: 88); // horizontal offset from starting signal ↗
43     wr16(addr: REG_HSYNC0, value: 0); // hsync falls
44     wr16(addr: REG_HSYNC1, value: 48); // hsync rise
45     wr16(addr: REG_VCYCLE, value: 525); // number of vertical cycles for display ↗
46     wr16(addr: REG_VOFFSET, value: 32); // vertical offset from start signal ↗
47     wr16(addr: REG_VSYNC0, value: 0); // vsync falls
48     wr16(addr: REG_VSYNC1, value: 3); // vsync rise
49     wr8(addr: REG_SWIZZLE, value: 0); // output swizzle, Inverse R et B
```

```
50 wr8(addr: REG_PCLK_POL, value: 0); // Règle la polarité du clock
51 wr8(addr: REG_CSPREAD, value: 0); // output clock spread enable
52 wr16(addr: REG_HSIZE, value: 800); // width resolution
53 wr16(addr: REG_VSIZE, value: 480); // height resolution
54 wr8(addr: REG_DITHER, value: 1); // output number of bits
55
56 // Nettoie l'écran et le met en rouge
57
58 wr32(addr: RAM_DL + 0, value: CLEAR_COLOR_RGB(255, 0, 0));
59 wr32(addr: RAM_DL + 4, value: CLEAR(1, 1, 1));
60 int offset = 8;
61 for (i = 0; i < 16; i++) {
62     wr32(addr: RAM_DL + offset, value: BITMAP_HANDLE(i));
63     offset += 4;
64     wr32(addr: RAM_DL + offset, value: BITMAP_LAYOUT_H(0, 0));
65     offset += 4;
66     wr32(addr: RAM_DL + offset, value: BITMAP_SIZE_H(0, 0));
67     offset += 4;
68 }
69
70 wr32(addr: RAM_DL + offset, value: DISPLAY());
71 wr8(addr: REG_DLSWAP, value: DLSWAP_FRAME);
72
73 wr8(addr: REG_GPIO_DIR, value: 0x80); // Met GPIO7 en sortie
74 wr8(addr: REG_GPIO, value: 0x80); // allume le backlight (met GPIO7 à 1)
75
76 wr8(addr: REG_PCLK, value: 2); // Après cette instruction, l'écran s'allume
77 }
78
79 void HostCommand(uint8_t command) {
80     SPI_CS_TFTStateSet(0);
81
82     tr8(command);
83     tr8(0);
84     tr8(0);
85
86     SPI_CS_TFTStateSet(1);
87 }
88
89 void wr8(uint32_t addr, uint8_t value) {
90     SPI_CS_TFTStateSet(0);
91
92     tr8(0x80 | (addr >> 16));
93     tr8(addr >> 8);
94     tr8(addr);
95     tr8(value);
96
97     SPI_CS_TFTStateSet(1);
98 }
99
100 void wr16(uint32_t addr, uint16_t value) {
```

```
101     SPI_CS_TFTStateSet(0);
102
103     tr8(0x80 | (addr >> 16));
104     tr8(addr >> 8);
105     tr8(addr);
106     tr16(value);
107
108     SPI_CS_TFTStateSet(1);
109 }
110
111 void wr32(uint32_t addr, uint32_t value) {
112     SPI_CS_TFTStateSet(0);
113
114     tr8(0x80 | (addr >> 16));
115     tr8(addr >> 8);
116     tr8(addr);
117     tr32(value);
118
119     SPI_CS_TFTStateSet(1);
120 }
121
122 uint8_t wr8s(uint32_t addr, const char *S) {
123     int l, i;
124     SPI_CS_TFTStateSet(0);
125
126     tr8(0x80 | (addr >> 16));
127     tr8(addr >> 8);
128     tr8(addr);
129
130     l = strlen((const char *) S);
131     for (i = 0; i <= l; i++) {
132         tr8(S[i]);
133     }
134     for (; i % 4 > 0; i++) {
135         tr8(0);
136     }
137
138     SPI_CS_TFTStateSet(1);
139     return i;
140 }
141
142 uint8_t rd8(uint32_t addr) {
143     SPI_CS_TFTStateSet(0);
144
145     tr8(addr >> 16 & 0x3F);
146     tr8(addr >> 8);
147     tr8(addr);
148     tr8(0); // dummy byte
149
150     uint8_t LSB = spi_read1(val: 0);
151
152     SPI_CS_TFTStateSet(1);
153     return LSB;
```

```
154 }
155
156 uint16_t rd16(uint32_t addr) {
157     SPI_CS_TFTStateSet(0);
158
159     tr8(addr >> 16 & 0x3F);
160     tr8(addr >> 8);
161     tr8(addr);
162     tr8(0); // dummy byte
163     uint8_t LSB = spi_read1(Val: 0);
164     uint8_t MSB = spi_read1(Val: 0);
165     SPI_CS_TFTStateSet(1);
166     return (MSB << 8) | LSB;
167 }
168
169 uint32_t rd32(uint32_t addr) {
170     SPI_CS_TFTStateSet(0);
171     tr8(addr >> 16 & 0x3F);
172     tr8(addr >> 8);
173     tr8(addr);
174     tr8(0); // dummy byte
175     uint8_t Byte0 = spi_read1(Val: 0);
176     uint8_t Byte1 = spi_read1(Val: 0);
177     uint8_t Byte2 = spi_read1(Val: 0);
178     uint8_t Byte3 = spi_read1(Val: 0);
179     SPI_CS_TFTStateSet(1);
180     return (Byte0 << 24) | (Byte1 << 16) | (Byte2 << 8) | Byte3;
181 }
182
183 void cmd(uint32_t command) {
184     wr32(addr: RAM_CMD + cmd_offset, value: command);
185     cmd_incrementn(n: 4);
186 }
187
188 void dl(uint32_t instruction) {
189     wr32(addr: RAM_DL + dl_offset, value: instruction);
190     dl_incrementn(n: 4);
191 }
192
193 void dl_incrementn(unsigned char n) {
194     dl_offset = (dl_offset + n) % 4096;
195 }
196
197 void cmd_incrementn(unsigned char n) {
198     cmd_offset = (cmd_offset + n) % 4096;
199 }
200
```